LASER SECURITY SYSTEM

Hardware Documentation & Source Code

CSCE 5612 – Embedded Hardware/Software Design | Spring 2026

University of North Texas

Milestone 3 Final Submission | Due: April 28, 2026

1. System Overview

Laser Security System represents an IoT security application designed for implementation using the Raspberry Pi Pico W board. The application incorporates the use of a laser beam from a KY-008 laser module along with an LDR photoresistor in order to develop a laser trip wire functionality. In case the laser beam is interrupted by any intruder, the alarm is triggered locally using LEDs and a buzzer, and simultaneously the live alert is sent to the Blynk IoT cloud dashboard and via an email notification using the EmailJS API.

2. Hardware Components

Component	Model / Spec	GPIO Pin	Function
Microcontroller	Raspberry Pi Pico W	—	Main MCU + Wi-Fi (802.11 b/g/n)
Laser Module	KY-008	GP2 (OUT)	Transmits laser beam
LDR Sensor	GL5528 Photoresistor	GP26 / ADC0	Receives/detects laser beam
LED 1 (PWM)	5mm Red LED	GP4 (PWM)	Visual alarm indicator
LED 2	5mm Red LED	GP5 (OUT)	Secondary visual indicator
Buzzer	Active Piezo Buzzer	GP6 (OUT)	Audible alarm output
Custom PCB Shield	JLCPCB 2-layer FR-4	—	Signal routing, headers, power rails
Power Supply	USB 5V / 3.3V rail	VSYS / 3V3	Powers all components

3. Custom PCB Shield

3.1 PCB Specifications

Parameter	Value
Manufacturer	JLCPCB
Board Dimensions	111.76 mm × 129.54 mm (4.4 × 5.1 inches)
Layers	2-layer FR-4
Base Material	FR-4 (Standard)
Surface Finish	HASL (Lead-free)
PCB Quantity Ordered	5 units
Build Time	2 days
PCB Cost	$9.80 (excl. shipping)
Engineering Fee	$4.00
Board Cost	$5.80
Shipping (DHL Express)	$29.56 (2–4 business days)
Product Type	Industrial / Consumer Electronics

3.2 PCB Design Purpose

Guarantees consistent header connections to all sensors and actuators. Solves the problem of breadboard wire reliability experienced during the implementation. Ensures power rail delivery of 3.3V and GND. Has pads specifically for the Laser sensor, LDR resistor, buzzer, and LEDs. Fabricated using KiCad software; then the Gerber file was created and sent to JLCPCB.

4. GPIO Pin Assignment

GPIO Pin	MicroPython Name	Type	Connected To
GP2	Laser	Digital OUT	KY-008 Laser Module
GP26 (ADC0)	LDR	Analog IN	LDR Photoresistor (voltage divider)
GP4	LED1 (PWM)	PWM OUT	LED 1 (brightness control)
GP5	LED2	Digital OUT	LED 2 (on/off)
GP6	Buzzer	Digital OUT	Active Piezo Buzzer
VSYS	—	Power IN	USB 5V input
3V3	—	Power OUT	3.3V rail for sensors
GND	—	Ground	Common ground

5. System Architecture

The system follows a polling-based embedded loop with event-driven cloud dispatch:

This is how the system works:

- There is a constant emission of the laser (KY-008) towards the Light-Dependent Resistor (LDR).
- The LDR ADC value is read every 100 milliseconds through the function LDR.read_u16().
- There is a threshold value of 30,000. Any values above this will be an indication that the beam is interrupted, indicating an intruder.
- On detecting an intruder, the system triggers an alert by blinking the LEDs three times and activating the buzzer.
- Once there is a change of state, the system updates the Blynk application, which sets the value of the virtual pin V0 to 255.
- Finally, the EmailJS REST API sends an HTTP POST request to trigger a Gmail alert.
- The last_state value is used to avoid repeated calls to the cloud API.

6. IoT & Cloud Integration

6.1 Wi-Fi Connectivity

- Protocol: IEEE 802.11 b/g/n via Pico W onboard module
- Library: MicroPython network module (STA_IF mode)
- Retry loop: 20 attempts × 500ms = 10s timeout

6.2 Blynk IoT Dashboard

Parameter	Value
Auth Token	CbrMAhk5SGxdKzVpwSyLKj7Rt7P8lCAh
Virtual Pin	V0 (LED widget)
ON State	255 (full brightness red LED)
OFF State	0 (LED off)
API Endpoint	https://blynk.cloud/external/api/update
Messages Used	71 of 100,000 (free tier)

6.3 EmailJS Email Alerts

Parameter	Value
Service ID	service_v1dm6u8 (Gmail backend)

Parameter	Value
Template ID	template_h9uemrb
User ID	Fb4-GX4zWxIPK1QbK
Alert Email	udayasairamineni01@gmail.com
API Endpoint	https://api.emailjs.com/api/v1.0/email/send
Origin Header	http://localhost (bypasses 403 on free plan)
Requests Used	7 of 200 (resets May 21)

7. Source Code – main.py

```
from machine import Pin, PWM, ADC
import network
import urequests
import ujson
from time import sleep

WIFI_SSID = "SpectrumSetup-B73A"
WIFI_PASSWORD = "westoffice499"

EMAILJS_SERVICE_ID = "service_v1dm6u8"
EMAILJS_TEMPLATE_ID = "template_h9uemrb"
EMAILJS_USER_ID = "Fb4-GX4zWxIPK1QbK"
ALERT_EMAIL = "udayasairamineni01@gmail.com"

BLYNK_AUTH = "CbrMAhk5SGxdKzVpwSyLKj7Rt7P8lCAh"
BLYNK_LED_PIN = "V0"

Laser = Pin(2, Pin.OUT)
LDR = ADC(26)
LED1 = PWM(Pin(4))
LED2 = Pin(5, Pin.OUT)
Buzzer = Pin(6, Pin.OUT)

Laser.value(1)
LED1.freq(1000)
threshold = 30000

def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(WIFI_SSID, WIFI_PASSWORD)
    print("Connecting to WiFi", end="")
    for _ in range(20):
        if wlan.isconnected():
            break
        print(".", end="")
        sleep(0.5)
    if wlan.isconnected():
        print("\nWiFi connected! IP:", wlan.ifconfig()[0])
    else:
```

```
        print("\nWiFi connection FAILED")

def send_email_alert():
    try:
        url = "https://api.emailjs.com/api/v1.0/email/send"
        payload = {
            "service_id": EMAILJS_SERVICE_ID,
            "template_id": EMAILJS_TEMPLATE_ID,
            "user_id": EMAILJS_USER_ID,
            "template_params": {
                "to_email": ALERT_EMAIL,
                "subject": "INTRUDER ALERT!",
                "message": "Motion detected! Laser beam broken at your premises!"
            }
        }
        headers = {
            "Content-Type": "application/json",
            "origin": "http://localhost"
        }
        res = urequests.post(url, data=ujson.dumps(payload), headers=headers)
        print("Email sent! Status:", res.status_code)
        res.close()
    except Exception as e:
        print("Email error:", e)

def blynk_set_led(state):
    try:
        url = "https://blynk.cloud/external/api/update?token={}&{}={}".format(
            BLYNK_AUTH, BLYNK_LED_PIN, state
        )
        res = urequests.get(url)
        print("Blynk LED:", "ON" if state == 255 else "OFF", "| Status:",
res.status_code)
        res.close()
    except Exception as e:
        print("Blynk error:", e)

def trigger_alarm():
    print("Triggering alarm - 3 blinks!")
    for i in range(3):
        LED1.duty_u16(65535)
        LED2.value(1)
        Buzzer.value(1)
        sleep(0.3)
        LED1.duty_u16(0)
        LED2.value(0)
        Buzzer.value(0)
        sleep(0.3)

def stop_alarm():
    Buzzer.value(0)
    LED2.value(0)
    LED1.duty_u16(20000)

connect_wifi()
sleep(1)

last_state = None
while True:
    value = LDR.read_u16()
    print("LDR:", value)
```

```
    if value > threshold:
        print("ALERT! Intruder detected!")
        trigger_alarm()
        if last_state != "alert":
            blynk_set_led(255)
            send_email_alert()
            last_state = "alert"
    else:
        if last_state != "clear":
            blynk_set_led(0)
            last_state = "clear"
            print("All clear!")
        stop_alarm()
    sleep(0.1)
```

8. Function Reference

Function	Returns	Description
connect_wifi()	None	Connects Pico W to Wi-Fi with 20-attempt retry loop
send_email_alert()	None	HTTP POST to EmailJS API; sends intruder alert email
blynk_set_led(state)	None	HTTP GET to Blynk API to set virtual pin V0 (0 or 255)
trigger_alarm()	None	Blinks LED1 + LED2 and activates buzzer 3 times
stop_alarm()	None	Turns off buzzer and LED2; dims LED1 to standby level

9. Testing & Results

Test Case	Result	Notes
Wi-Fi connection	PASS	Connects within 5s on first attempt
Laser beam clear (ADC < 30000)	PASS	All clear state, no alert triggered
Beam broken (ADC > 30000)	PASS	Alarm + Blynk + email fired correctly
Blynk LED ON/OFF	PASS	LED turns red on alert, off on clear
EmailJS alert delivery	PASS	Email received within ~2 seconds
Duplicate alert suppression	PASS	last_state prevents repeat API calls
Detection latency	< 150ms	From beam break to local alarm
False alarm rate	< 1%	After threshold tuning at 30000

10. Libraries & Dependencies

- MicroPython v1.22+ for Raspberry Pi Pico W

- machine – GPIO, PWM, ADC control
- network – Wi-Fi (STA_IF) connection management
- urequests – HTTP GET and POST requests
- ujson – JSON serialization for API payloads
- time.sleep – Timing and delay control
- Blynk Cloud API – Virtual pin updates via HTTP REST
- EmailJS REST API v1.0 – Email dispatch without backend

11. Known Limitations & Future Work

- Reconnection with Wi-Fi occurs manually in case of lost signals (future scope: develop a reconnection loop automatically).
- EmailJS free package can send at most 200 messages per month.
- FreeRTOS is not utilized; bare metal programming in cooperation mode is used instead.
- A one-grid laser is employed in the present design, while future scope includes multiple grids.
- No camera module is provided for recording images of the intruder.
- A battery backup has not been implemented yet for better reliability.